

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 1

INFORME DE LABORATORIO

(formato estudiante)

INFORMACIÓN BÁSICA					
ASIGNATURA:	COMPUTACIÓN GRAFICA, VISIÓN COMPUTACIONAL Y MULTIMEDIA (E)				
TÍTULO DE LA PRÁCTICA:	<i>Clasificación y Reconocimiento</i>				
NÚMERO DE PRÁCTICA:	<i>09</i>	AÑO LECTIVO:	<i>5to</i>	NRO. SEMESTRE:	<i>NOVENO</i>
FECHA DE PRESENTACIÓN	<i>dd/mm/aaaa</i>	HORA DE PRESENTACIÓN	<i>hh/mm/ss</i>		
INTEGRANTE (s) <i>Sencia Ale Bryan Daniel</i>				NOTA (0-20)	
DOCENTE(s): MBA Mg. Ing. RENE ALONSO NIETO VALENCIA.					

RESULTADOS Y PRUEBAS	
Link del repositorio: https://github.com/SenciaAleBryanDaniel/Lab_CGVCM.git	
Índice.	
I. Configuración del Entorno y Adquisición de Video	2
II. Segmentación y Reconocimiento de Pose	3
III. Cálculo de Ángulos Articulares	5
IV. Lógica de Estado y Contador (Prueba con Curl de Bíceps)	8
V. Implementación Final (Contador de Sentadillas)	11
VI. Comparación de Tecnologías	16

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 2

I. Configuración del Entorno y Adquisición de Video

Explicación.

Se aisló el entorno de trabajo usando Python 3.11, ya que versiones más recientes no cuentan con binarios estables para las librerías de visión artificial. Dentro de un entorno virtual (venv) se instalaron OpenCV y NumPy, y se probó la captura de cámara con un script básico antes de integrar el modelo de reconocimiento.

Comandos Usados.

```
# Creación del entorno forzando la versión estable de Python
py -3.11 -m venv venv

# Activación del entorno en Windows PowerShell
venv\Scripts\activate

# Instalación de librerías base
pip install opencv-python numpy
```

Código usado (test_camara.py).

```
import cv2

cap = cv2.VideoCapture(0)

while True:
    # Captura frame por frame
    ret, frame = cap.read()
    if not ret:
        break

    # Visualización de los datos capturados
    cv2.imshow('Prueba Camara - Proyecto', frame)

    # Condición de salida limpia
    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 3

Análisis del código.

cv2.VideoCapture(0) abre la cámara web. El bucle lee frame por frame con cap.read(), los muestra con cv2.imshow(), y termina al presionar 'q'. Al final se libera la cámara y se cierran las ventanas.

Resultados.



Img. 01 - Terminal mostrando el prefijo (venv) activo en la ruta del proyecto.



Img. 02 - Ventana "Prueba Camara - Proyecto" mostrando la captura en tiempo real.

II. Segmentación y Reconocimiento de Pose

Explicación.

Se integró MediaPipe Pose para detectar la topología del cuerpo humano. Con la versión más reciente de la librería aparecieron errores AttributeError y ModuleNotFoundError, ya que versiones superiores a la 0.10.31 deprecaron la API tradicional de solutions. Se solucionó fijando las dependencias en mediapipe==0.10.5 y protobuf==3.20.3. Como MediaPipe requiere imágenes en RGB y OpenCV captura en BGR, cada frame se convierte antes de procesarse; el modelo extrae 33 landmarks (x, y, z) y dibuja el esqueleto sobre el video.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 4

Comandos usados.

```
# Limpieza de dependencias conflictivas
pip uninstall mediapipe protobuf -y
# Instalación de la dupla estable
pip install mediapipe==0.10.5 protobuf==3.20.3
```

Código usado.

```
import cv2
import mediapipe as mp

mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(min_detection_confidence=0.5,
min_tracking_confidence=0.5)

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    # MediaPipe requiere RGB, OpenCV captura en BGR
    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    resultado = pose.process(frame_rgb)

    if resultado.pose_landmarks:
        mp_drawing.draw_landmarks(
            frame, resultado.pose_landmarks, mp_pose.POSE_CONNECTIONS
        )

    cv2.imshow('Deteccion de Pose', frame)

    if cv2.waitKey(1) & 0xFF == ord('q'):
        break

cap.release()
cv2.destroyAllWindows()
```

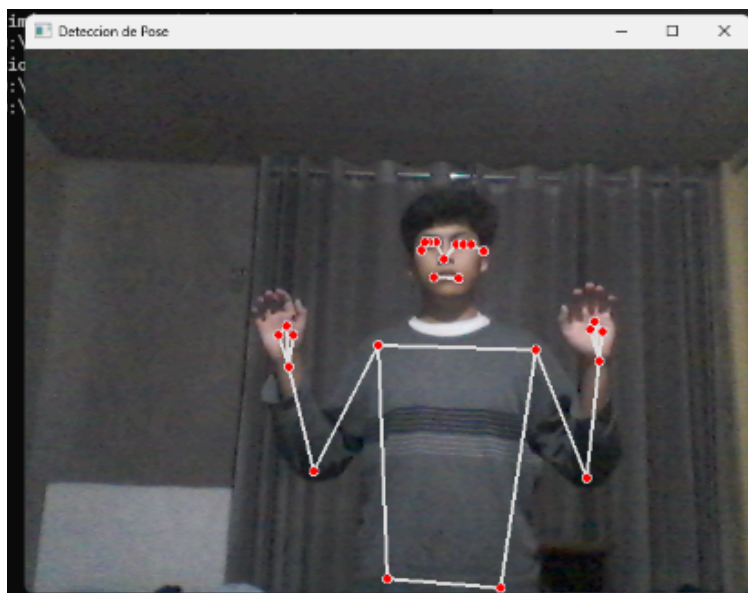
Explicación del código.

mp_pose.Pose() configura el modelo con umbrales de confianza del 50%. cv2.cvtColor() convierte el frame a RGB antes de pasarlo a pose.process(), que devuelve las coordenadas de los 33 landmarks si

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 5

detecta una persona. draw_landmarks() dibuja los puntos y las conexiones (POSE_CONNECTIONS) sobre el frame original en BGR.

Resultados.



Img. 03 - Extracción exitosa de coordenadas topológicas usando MediaPipe Pose: esqueleto trazado sobre el cuerpo en tiempo real.

III. Cálculo de Ángulos Articulares

Explicación.

Para cuantificar el movimiento del brazo se implementó una función que calcula el ángulo interno formado por tres landmarks: hombro (punto 11), codo como vértice (punto 13) y muñeca (punto 15), todos del lado izquierdo. El ángulo se obtiene mediante la diferencia de arcotangentes de los vectores hombro-codo y muñeca-codo, y se normaliza para mantenerse en un rango de 0° a 180°, coherente con la flexión anatómica real del brazo. El valor resultante se muestra en tiempo real sobre la posición del codo en el video.

Sustento Matemático.

Se definen tres puntos: A (hombro), B (codo, vértice) y C (muñeca). El ángulo interno θ se calcula como la diferencia entre los ángulos que forman los vectores $B \rightarrow C$ y $B \rightarrow A$ respecto al eje horizontal:

$$\theta = | \arctan2(y_C - y_B, x_C - x_B) - \arctan2(y_A - y_B, x_A - x_B) | \times (180 / \pi)$$

 Si θ supera 180°, se aplica $\theta = 360^\circ - \theta$, para mantener el resultado dentro del rango anatómicamente válido (0° a 180°).

Código usado (pose_basico.py).

```
import cv2
import mediapipe as mp
import numpy as np
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 6

```

def calcular_angulo(a, b, c):
    a = np.array(a) # Hombro
    b = np.array(b) # Codo (vertice)
    c = np.array(c) # Muneca

    radianes = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1],
a[0]-b[0])
    angulo = np.abs(radianes * 180.0 / np.pi)

    if angulo > 180.0:
        angulo = 360.0 - angulo

    return angulo

mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(min_detection_confidence=0.5,
min_tracking_confidence=0.5)

cap = cv2.VideoCapture(0)

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    resultado = pose.process(frame_rgb)

    if resultado.pose_landmarks:
        landmarks = resultado.pose_landmarks.landmark

        hombro = [landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].y]
        codo = [landmarks[mp_pose.PoseLandmark.LEFT_ELBOW.value].x,
landmarks[mp_pose.PoseLandmark.LEFT_ELBOW.value].y]
        muneca = [landmarks[mp_pose.PoseLandmark.LEFT_WRIST.value].x,
landmarks[mp_pose.PoseLandmark.LEFT_WRIST.value].y]

        angulo = calcular_angulo(hombro, codo, muneca)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 7

```

h, w, _ = frame.shape
coordenada_texto = tuple(np.multiply(codo, [w, h]).astype(int))

cv2.putText(frame, str(int(angulo)), coordenada_texto,
            cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2,
cv2.LINE_AA)

mp_drawing.draw_landmarks(frame, resultado.pose_landmarks,
mp_pose.POSE_CONNECTIONS)

cv2.imshow('Calculo Geometrico - Curl de Biceps', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

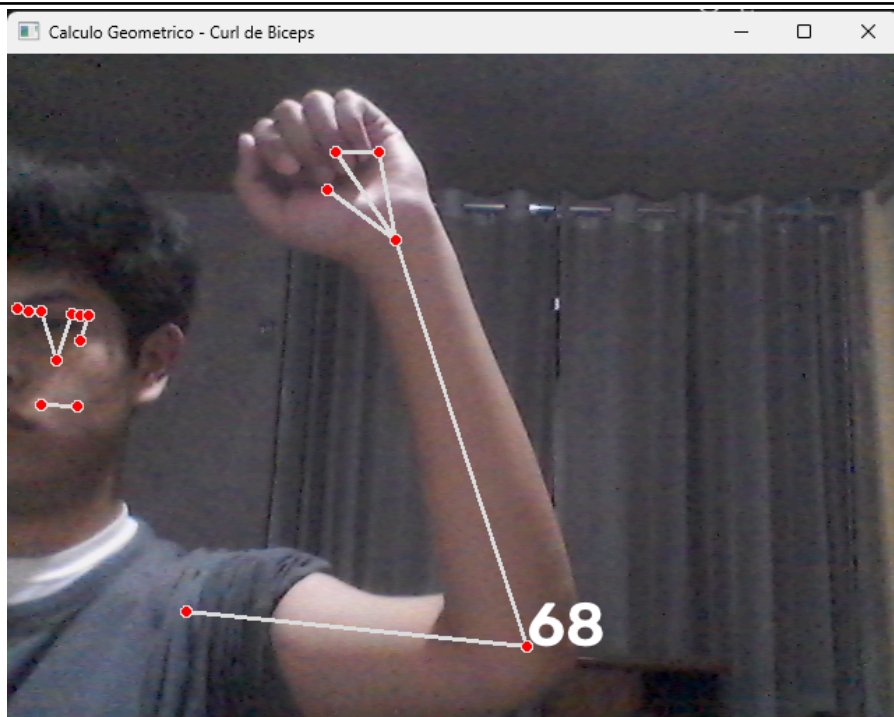
cap.release()
cv2.destroyAllWindows()

```

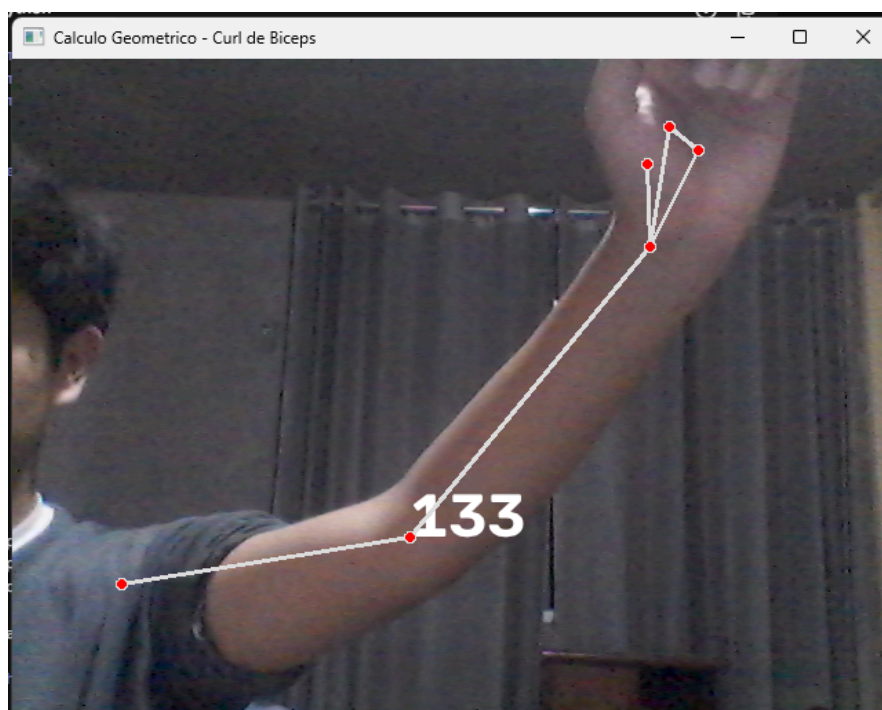
Análisis del código.

La función `calcular_angulo()` convierte los tres puntos en arreglos NumPy y calcula el ángulo entre los vectores `codo→hombro` y `codo→muñeca` usando `np.arctan2()`, que da el ángulo de cada vector respecto al eje horizontal; la resta entre ambos da el ángulo entre ellos. El resultado se convierte de radianes a grados y se ajusta con `360 - angulo` si excede 180° , para mantenerlo en el rango anatómico correcto. Los landmarks se extraen normalizados (entre 0 y 1), por lo que `np.multiply(codo, [w, h])` los reescala a coordenadas de píxeles reales para ubicar el texto del ángulo directamente sobre el codo en la imagen.

Resultados.



Img. 04 - Brazo extendido, mostrando un ángulo cercano a 68° sobre el codo.



Img. 05 - Brazo flexionado, mostrando un ángulo cercano a 133° sobre el codo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 9

IV. Lógica de Estado y Contador (Prueba con Curl de Bíceps)

Explicación.

Antes de implementar el contador definitivo de sentadillas (Fase 5), se probó la lógica de conteo usando el ángulo del brazo ya calculado en la fase anterior. Para esto se definieron dos variables de estado: contador (entero que inicia en 0) y etapa (texto que indica si el brazo está "abajo" o "arriba"). Se implementó una máquina de estados simple: el sistema solo incrementa el contador si detecta que el ángulo pasó de un estado extendido ($>160^\circ$) a uno flexionado ($<40^\circ$), evitando conteos falsos por fluctuaciones momentáneas de la detección. Además, se agregó una interfaz gráfica (HUD) sobre el video, mostrando en tiempo real el número de repeticiones y la etapa actual.

Sustento Lógico.

Se definieron dos umbrales sobre el ángulo θ calculado previamente:

- Si $\theta > 160^\circ$, se fija etapa = "abajo" (brazo extendido).
- Si $\theta < 40^\circ$ y la etapa previa era "abajo", se actualiza etapa = "arriba" y se incrementa el contador.

Esta condición obliga a que exista una transición completa (abajo $\rightarrow$ arriba) antes de contar una repetición, en lugar de contar cada vez que el ángulo cruza un solo umbral.

Código usado (prueba, pose_basico.py).

```
import cv2
import mediapipe as mp
import numpy as np

def calcular_angulo(a, b, c):
    a = np.array(a)
    b = np.array(b)
    c = np.array(c)

    radianes = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1],
a[0]-b[0])
    angulo = np.abs(radianes * 180.0 / np.pi)

    if angulo > 180.0:
        angulo = 360.0 - angulo

    return angulo

mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(min_detection_confidence=0.5,
min_tracking_confidence=0.5)

cap = cv2.VideoCapture(0)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 10

```

# Variables de estado y conteo
contador = 0
etapa = None

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    resultado = pose.process(frame_rgb)

    if resultado.pose_landmarks:
        landmarks = resultado.pose_landmarks.landmark

        hombro = [landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].x,
                  landmarks[mp_pose.PoseLandmark.LEFT_SHOULDER.value].y]
        codo = [landmarks[mp_pose.PoseLandmark.LEFT_ELBOW.value].x,
               landmarks[mp_pose.PoseLandmark.LEFT_ELBOW.value].y]
        muneca = [landmarks[mp_pose.PoseLandmark.LEFT_WRIST.value].x,
                 landmarks[mp_pose.PoseLandmark.LEFT_WRIST.value].y]

        angulo = calcular_angulo(hombro, codo, muneca)

        # Logica del contador (maquina de estados)
        if angulo > 160:
            etapa = "abajo"
        if angulo < 40 and etapa == "abajo":
            etapa = "arriba"
            contador += 1

        h, w, _ = frame.shape
        coordenada_texto = tuple(np.multiply(codo, [w, h]).astype(int))
        cv2.putText(frame, str(int(angulo)), coordenada_texto,
                    cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2,
cv2.LINE_AA)

        mp_drawing.draw_landmarks(frame, resultado.pose_landmarks,
mp_pose.POSE_CONNECTIONS)

        # Interfaz grafica (marcador en pantalla)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 11

```

cv2.rectangle(frame, (0,0), (250,80), (245,117,16), -1)

cv2.putText(frame, 'REPS', (15,20), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0,0,0), 1, cv2.LINE_AA)
cv2.putText(frame, str(contador), (10,65), cv2.FONT_HERSHEY_SIMPLEX,
2, (255,255,255), 2, cv2.LINE_AA)

cv2.putText(frame, 'ETAPA', (100,20), cv2.FONT_HERSHEY_SIMPLEX, 0.6,
(0,0,0), 1, cv2.LINE_AA)
cv2.putText(frame, etapa if etapa else '-', (100,65),
cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255,255,255), 2, cv2.LINE_AA)

cv2.imshow('Contador Inteligente', frame)

if cv2.waitKey(1) & 0xFF == ord('q'):
    break

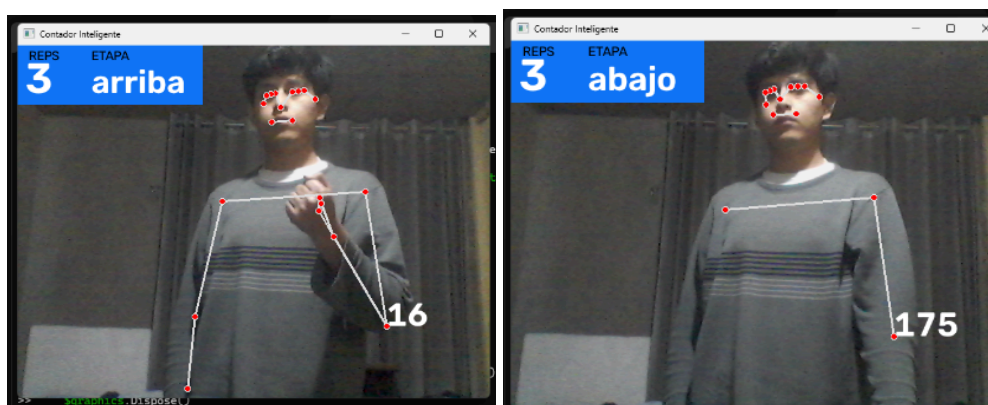
cap.release()
cv2.destroyAllWindows()

```

Análisis del código.

Las variables contador y etapa se declaran fuera del bucle while para que conserven su valor entre frames (persistencia de estado). Dentro del bucle, las dos condicionales if actúan como una máquina de estados de dos posiciones: solo se permite pasar a "arriba" e incrementar el contador si el estado previo era "abajo", evitando que fluctuaciones del ángulo alrededor de un solo umbral generen conteos repetidos. Para la interfaz, cv2.rectangle() dibuja un panel sólido (grosor -1) en la esquina superior izquierda, y cv2.putText() superpone las etiquetas "REPS" y "ETAPA" junto con sus valores actuales, actualizándose en cada frame.

Resultados.



Img. 06 - HUD mostrando el contador en 3 repeticiones y la etapa "arriba", tras completar tres flexiones de brazo frente a la cámara.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 12

V. Implementación Final (Contador de Sentadillas)

Explicación.

Validada la lógica de estados en el brazo, se adaptó el sistema para rastrear la pierna izquierda, reemplazando Hombro/Codo/Muñeca por Cadera (punto 23), Rodilla (punto 25, como vértice) y Tobillo (punto 27). Se recalibraron los umbrales de la máquina de estados para el rango de movimiento de una sentadilla: el estado de reposo ("arriba") corresponde a la persona de pie con la pierna extendida ($\theta > 160^\circ$), y el estado de flexión ("abajo") se activa al romper el plano paralelo de la rodilla ($\theta < 90^\circ$). A diferencia del brazo, aquí el contador se incrementa al regresar a la posición de pie (no al bajar), garantizando que solo se cuenten repeticiones con rango de movimiento completo.

Sustento Lógico.

Si $\theta < 90^\circ$, se fija etapa = "abajo" (sentadilla completa).

Si $\theta > 160^\circ$ y la etapa previa era "abajo", se actualiza etapa = "arriba" y se incrementa el contador. Esto exige que el ciclo completo (de pie $\rightarrow$ agachado $\rightarrow$ de pie) se cumpla antes de registrar una repetición válida, evitando falsos positivos por movimientos parciales.

Código usado (versión final, pose_basico.py).

```
import cv2
import mediapipe as mp
import numpy as np

def calcular_angulo(a, b, c):
    a = np.array(a)
    b = np.array(b)
    c = np.array(c)

    radianes = np.arctan2(c[1]-b[1], c[0]-b[0]) - np.arctan2(a[1]-b[1],
a[0]-b[0])
    angulo = np.abs(radianes * 180.0 / np.pi)

    if angulo > 180.0:
        angulo = 360.0 - angulo

    return angulo

mp_pose = mp.solutions.pose
mp_drawing = mp.solutions.drawing_utils
pose = mp_pose.Pose(min_detection_confidence=0.5,
min_tracking_confidence=0.5)

cap = cv2.VideoCapture(0)
```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 13

```

contador = 0
etapa = None

while True:
    ret, frame = cap.read()
    if not ret:
        break

    frame_rgb = cv2.cvtColor(frame, cv2.COLOR_BGR2RGB)
    resultado = pose.process(frame_rgb)

    if resultado.pose_landmarks:
        landmarks = resultado.pose_landmarks.landmark

        # Extraccion de coordenadas de la pierna izquierda
        cadera = [landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].x,
                  landmarks[mp_pose.PoseLandmark.LEFT_HIP.value].y]
        rodilla = [landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].x,
                  landmarks[mp_pose.PoseLandmark.LEFT_KNEE.value].y]
        tobillo = [landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].x,
                  landmarks[mp_pose.PoseLandmark.LEFT_ANKLE.value].y]

        angulo = calcular_angulo(cadera, rodilla, tobillo)

        # Logica del contador de sentadillas
        if angulo > 160:
            if etapa == "abajo":
                contador += 1
                etapa = "arriba"

        if angulo < 90:
            etapa = "abajo"

        h, w, _ = frame.shape
        coordenada_texto = tuple(np.multiply(rodilla, [w,
h])).astype(int))
        cv2.putText(frame, str(int(angulo)), coordenada_texto,
                    cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255, 255, 255), 2,
cv2.LINE_AA)

```

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 14

```

        mp_drawing.draw_landmarks(frame, resultado.pose_landmarks,
        mp_pose.POSE_CONNECTIONS)

        # Interfaz grafica (marcador en pantalla)
        cv2.rectangle(frame, (0,0), (250,80), (245,117,16), -1)

        cv2.putText(frame, 'SENTADILLAS', (15,20), cv2.FONT_HERSHEY_SIMPLEX,
        0.5, (0,0,0), 1, cv2.LINE_AA)
        cv2.putText(frame, str(contador), (10,65), cv2.FONT_HERSHEY_SIMPLEX,
        2, (255,255,255), 2, cv2.LINE_AA)

        cv2.putText(frame, 'ESTADO', (130,20), cv2.FONT_HERSHEY_SIMPLEX, 0.5,
        (0,0,0), 1, cv2.LINE_AA)
        cv2.putText(frame, etapa if etapa else '-', (130,65),
        cv2.FONT_HERSHEY_SIMPLEX, 1.5, (255,255,255), 2, cv2.LINE_AA)

        cv2.imshow('Proyecto Final - Contador de Sentadillas', frame)

        if cv2.waitKey(1) & 0xFF == ord('q'):
            break

    cap.release()
    cv2.destroyAllWindows()

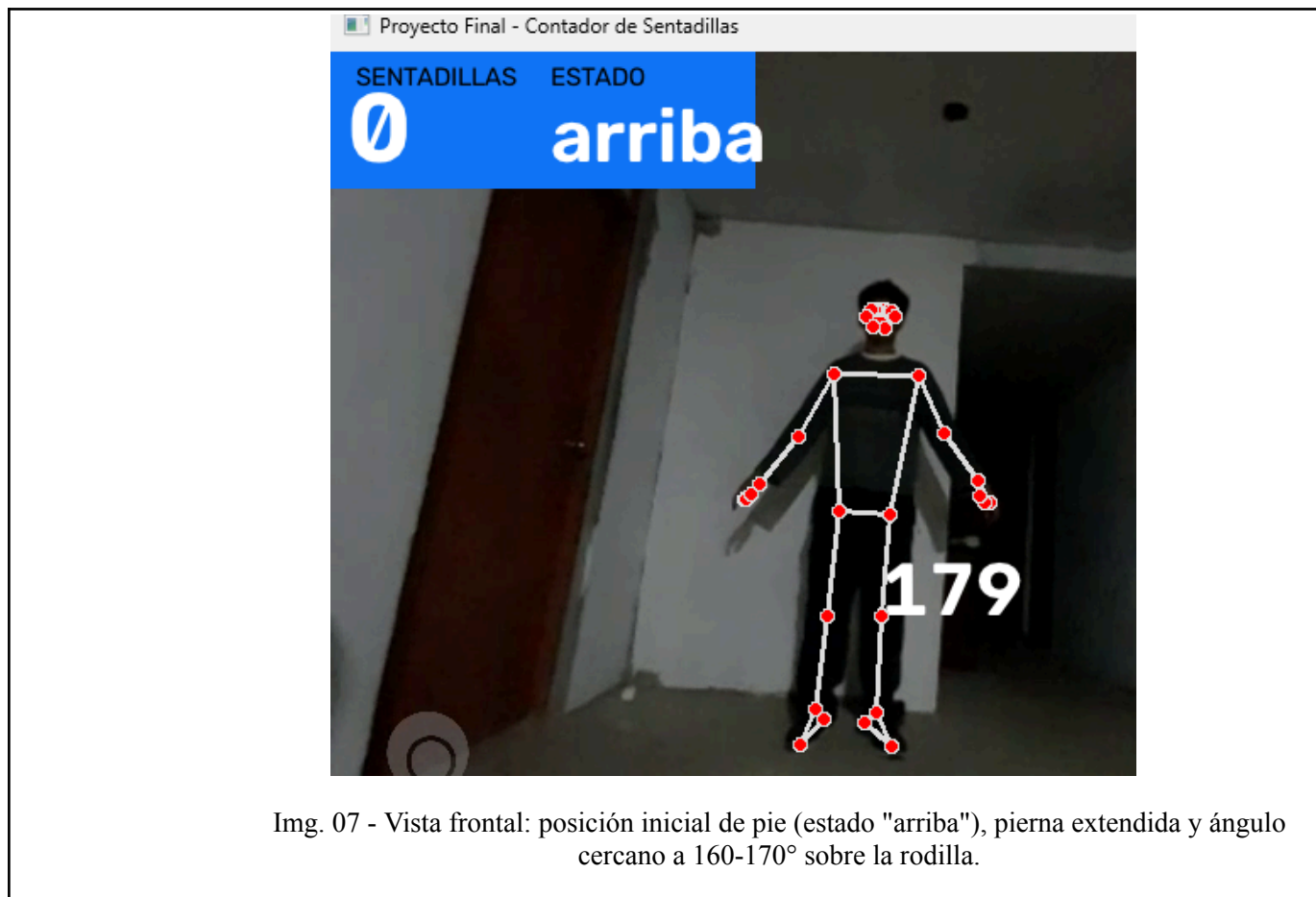
```

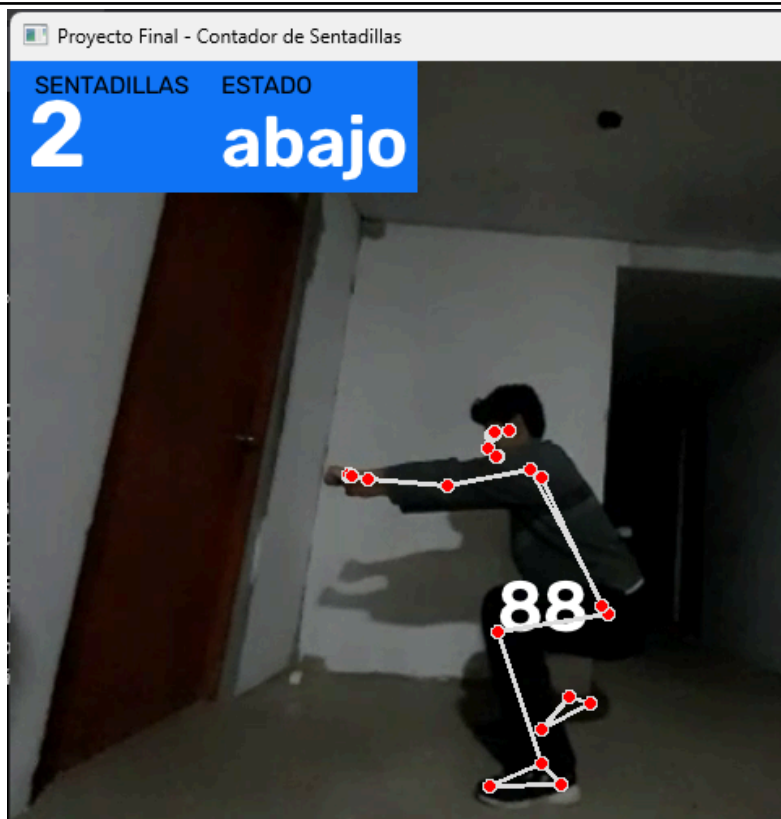
Análisis del código.

La estructura conserva la misma máquina de estados de la fase anterior, pero invierte el punto de disparo: el contador se incrementa dentro del bloque if ángulo > 160, y solo si la etapa previa era "abajo", es decir, al completar el regreso a la posición de pie tras una flexión válida. Esto evita contar el simple hecho de pararse sin haberse agachado antes. El resto de la lógica (extracción de landmarks, cálculo del ángulo y renderizado del HUD) es idéntico al usado para el brazo, cambiando únicamente los puntos anatómicos de referencia.

Resultados.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 15





Img. 08 - Vista lateral: posición en cuclillas (estado "abajo"), ángulo menor a 90° sobre la rodilla.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 17



Img. 09 - Vista lateral: regreso a la posición de pie, contador incrementado a 1 repetición y estado "arriba".

VI. Comparación de Tecnologías

MediaPipe Pose frente a OpenPose

El framework de MediaPipe destaca por su arquitectura ligera y su capacidad para realizar inferencia de modelos topológicos en tiempo real utilizando exclusivamente la CPU del equipo. OpenPose representa un estándar en precisión pero exige hardware dedicado y tarjetas gráficas de alto rendimiento para alcanzar la misma tasa de fotogramas por segundo, lo cual lo hace inviable para este prototipo ligero.

OpenCV frente a procesadores de imagen básicos

La librería OpenCV proporciona un motor optimizado para la manipulación matricial de los fotogramas y la interacción directa con los controladores de video del sistema operativo. Otras alternativas de procesamiento como Pillow carecen de la velocidad y las herramientas de renderizado necesarias para analizar y superponer gráficos sobre un flujo de video continuo.

Python frente a C++ en visión artificial

El uso de Python con entornos virtuales permite una orquestación ágil de las dependencias y una rápida iteración para programar la máquina de estados lógicos. El desarrollo del mismo pipeline en C++ ofrecería un rendimiento de ejecución superior pero incrementaría drásticamente la complejidad del código y el tiempo de compilación sin aportar una ventaja perceptible en la captura a treinta fotogramas por segundo.

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 18

CUESTIONARIO

1. ¿Describan un algoritmo utilizado para el reconocimiento y clasificación de objetos?

El algoritmo principal implementado es MediaPipe Pose. Este modelo analiza los fotogramas capturados y previamente convertidos al formato RGB para inferir la topología del cuerpo humano. El sistema logra extraer las coordenadas exactas de treinta y tres puntos clave articulares para luego trazar un esqueleto representativo sobre el video original.

2. ¿Qué problemas pueden ocurrir al realizar el reconocimiento y clasificación de objetos?

A nivel de software pueden presentarse graves conflictos de dependencias e incompatibilidades entre versiones que rompen la ejecución del programa. En cuanto a la captura visual pueden ocurrir fluctuaciones momentáneas en la detección espacial de los puntos, lo cual exige programar restricciones lógicas adicionales para evitar falsos positivos en el reconocimiento del movimiento.

CONCLUSIONES

El aislamiento del entorno de trabajo mediante herramientas virtuales y el control estricto de las versiones de las librerías son prácticas fundamentales para asegurar la estabilidad de un proyecto de visión artificial.

La transformación de las coordenadas topológicas extraídas en vectores bidimensionales permite cuantificar el movimiento humano de forma precisa aplicando trigonometría básica.

La integración de una máquina de estados finitos es una medida indispensable para validar transiciones de movimiento completas y rechazar variaciones de datos erróneas generadas por el ruido de la cámara.

METODOLOGÍA DE TRABAJO

REFERENCIAS Y BIBLIOGRAFÍA

	UNIVERSIDAD NACIONAL DE SAN AGUSTÍN FACULTAD DE INGENIERÍA DE PRODUCCIÓN Y SERVICIOS DEPARTAMENTO ACADÉMICO DE INGENIERÍA DE SISTEMAS E INFORMÁTICA	
Formato: Guía de Práctica de Laboratorio / Talleres / Centros de Simulación		
Aprobación: 2022/03/01	Código: GUIA-PRLE-001	Página: 19

[1] Gordon, V. S., & Clevenger, J. L. (2020). *Computer Graphics Programming in OpenGL with C++*.
[2] D. P. Kothari, G. Awari, D. Shrimankar, A. Bhende (2019), *Mathematics for Computer Graphics and Game Programming*.